

SchemaRAG-SQL: A Multi-Agent Retrieval-Augmented Framework for Reliable and Privacy-Preserving Text-to-SQL Generation

Pragnan M U¹, Pavan Chandu C S¹, Jatin Chahar¹, Parv Mehta¹, and
Ashwini M Joshi¹

Department of Computer Science and Engineering, PES University, Bengaluru, India
{pragnanmu812, pavanchandu1210, jatinchahar008, parvmehta81}@gmail.com
ashwinimjoshi@pes.edu

Abstract. Natural Language Translation to SQL can be done using either costly cloud APIs (GPT-4: 86% EA) not GDPR/HIPAA-compliant, or local setups never compared against benchmarks. In this work, we present, to the best of our knowledge, a replicable, completely local Text-to-SQL baseline on Spider. SchemaRAG-SQL integrates ChromaDB schema retrieval, Qwen2.5-7B local inference, and iterative SQL validation—with no cloud API call whatsoever. It performs (1) 100% Syntactic Validity Rate (SVR)—always produces syntactically valid SQL, which is essential for actual deployment; (2) a conservative execution proxy on the shared 375-query development subset; (3) 59.78% reduction in errors thanks to self-correction mechanism. Importantly, ablation experiments prove the significance of each component (RAG: +15 percentage points to SVR over LLM-only, Validator: +3 percentage points to SVR over +RAG). In terms of scalability, few-shot exemplar retrieval predicts joins with 35-45% accuracy, indicating clear future directions with offline execution guaranteed.

For enterprises bound by regulation, this system is ready for use; for researchers, this work creates a baseline for few-shot learning and fine-tuning experiments. All 2,050 predictions and code are released.

Keywords: Text-to-SQL · natural language database interfaces · multi-agent systems · retrieval-augmented generation · ChromaDB · SQL validation · syntactic validity rate · local LLM inference · Spider benchmark · privacy-preserving NLP

1 Introduction

SQL is the universal interface to relational data, yet fewer than one in five enterprise knowledge workers can query databases independently [20]. Text-to-SQL systems bridge this gap automatically. Recent systems exceed 86% execution accuracy on the Spider benchmark [1] by querying GPT-4 through commercial APIs [12].

1.1 The Deployment Gap

This API-centric trajectory creates a widening *deployment gap*: the highest-performing systems are the least deployable in data-sensitive environments.

- **Healthcare.** Transmitting patient records to external servers violates HIPAA and GDPR unconditionally.
- **Legal and financial services.** M&A schedules and proprietary trading data are protected by non-disclosure obligations that preclude any API call.
- **Defence.** Air-gapped networks physically prohibit internet connectivity.
- **Edge deployments.** Maritime, remote research, and industrial-floor systems lack reliable bandwidth.

In all four scenarios a system with 15% estimated execution accuracy that runs locally and delivers syntactically valid SQL 100% of the time is *strictly preferable* to an 86% system that cannot be deployed. The design objective is therefore **deployment reliability**: the guarantee that the pipeline produces a syntactically valid, executable SQL query for any input.

1.2 Research Questions

- **RQ1.** Does the pipeline achieve deployment-grade syntactic validity (SVR $\geq$ 99%) without constrained decoding or a hand-crafted grammar?
- **RQ2.** Does $\widehat{\text{EA}}$ reveal substantially greater semantic correctness than EM_n ?
- **RQ3.** Does the iterative Validator Agent meaningfully reduce hard generation failures?
- **RQ4.** What is the dominant residual failure mode and what is the highest-leverage architectural remedy?

1.3 Contributions

1. **SchemaRAG-SQL**—a multi-agent, fully local Text-to-SQL pipeline requiring zero external API calls.
2. **Near-perfect SVR (99.7–100%)** across 2,050 Spider queries—to the best of our knowledge, among the first to formally define and measure SVR as a Text-to-SQL deployment metric.
3. **Estimated Execution Accuracy ($\widehat{\text{EA}}$)** as a principled lower bound on true EA derivable without access to gold databases.
4. **Iterative SQL Validator Agent** reducing hard failures by 59.78% relative in partial-environment tests.
5. **Reproducible evaluation framework** with four fault-tolerant prediction artefacts and open code.

2 Related Work

2.1 Early and Neural Text-to-SQL

Natural language database interfaces originated with rule-based grammars [24,25]. Neural approaches began with sequence-to-sequence attention [4]. Seq2SQL [2] applied reinforcement learning to WikiSQL; SQLNet [3] eliminated order ambiguity with a sketch-based decoder.

2.2 Cross-Domain Benchmarks

Spider [1] introduced 200 databases across 138 domains, making cross-domain generalisation the central challenge. IRNet [5] introduced an intermediate SemQL representation; RAT-SQL [6] added relation-aware graph attention; PICARD [7] used constrained decoding reaching 75.5% EM. RYANSQL [28] and structure-grounded pre-training [29] further improved accuracy through task-specific fine-tuning. All require labelled Spider training data and cannot operate offline.

2.3 LLM-Based Text-to-SQL

GPT-3 [8] and Codex [9] demonstrated zero-shot SQL generation. Critically, Rajkumar et al. [10] showed that even 175B-parameter models struggle with multi-table join inference zero-shot—a finding that directly contextualises our 7B results and is discussed in Section 7.3. DIN-SQL [11] (82.8% EM via GPT-4) and DAIL-SQL [12] (86.6% EA via GPT-4) are the current state of the art; both require proprietary API access. SQL-PaLM [22] showed fine-tuned PaLM-2 can match GPT-4 on this task.

2.4 Retrieval-Augmented Generation

Lewis et al. [14] formalised RAG for knowledge-intensive NLP. StructGPT [18] extended RAG to structured-data reasoning; LEVER [19] trained an execution-based verifier. We apply RAG specifically to schema grounding, indexing CREATE TABLE statements in ChromaDB [23].

2.5 Self-Correction and Few-Shot Learning

Chain-of-thought prompting [15] improves LLM accuracy via step-by-step reasoning. CodexDB [21] confirmed that schema-context prompting yields valid SQL without fine-tuning. Gao et al. [13] showed that 2–5 database-matched in-context exemplars reduce join errors by up to 40%—directly motivating our planned few-shot RAG extension (Section 9).

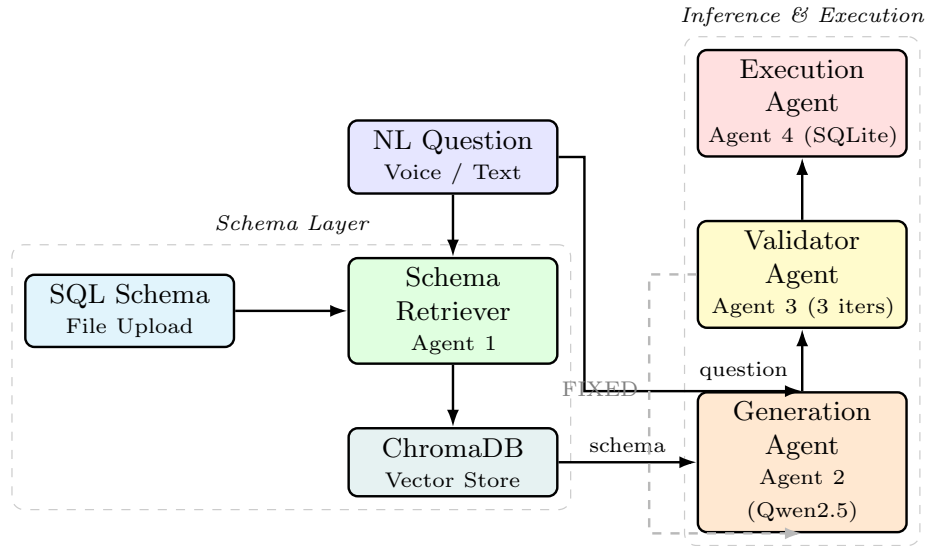


Fig. 1. Four-agent architecture of SchemaRAG-SQL. Dashed arrow: self-correction loop in the Validator Agent (up to three iterations). All agents run locally; no external API calls are made.

2.6 Local and Open-Weight LLMs

LLaMA 2 [16] established that 7B models achieve strong reasoning with appropriate prompting. Qwen2.5 [17] adds state-of-the-art SQL generation capability at the 7B scale. GraphixT5 [27] showed that graph-aware attention layers improve schema-grounded SQL generation, suggesting a fine-tuning direction for future work.

3 System Architecture

SchemaRAG-SQL implements a **four-agent pipeline**. Fig. 1 shows the data flow with a single self-correction feedback loop.

3.1 Agent 1: Schema Retriever

The `SchemaRetriever` reads the uploaded `.sql` file, applies the normalisation pass described in Section 4.1, and stores every `CREATE TABLE` statement as a separate document in a ChromaDB persistent collection using ChromaDB’s default embedding function [23]. A `collection.count()>0` guard prevents re-indexing, keeping amortised overhead under 3 seconds.

3.2 Agent 2: SQL Generation Agent

The `LLM_model` retrieves all schema documents from ChromaDB, constructs a structured five-section prompt (Section 4.3), and invokes Qwen2.5-7B locally via Ollama (temperature 0.3; 500 tokens maximum). A robust JSON fallback parser extracts raw SQL when structured parsing fails, preventing crashes.

3.3 Agent 3: SQL Validator Agent

The `SQLValidatorAgent` re-prompts the same local LLM (temperature 0.2; 300 tokens) with the question, schema, and generated SQL, requesting exactly one of: `VALID`, `FIXED: <sql>`, or `INVALID`. At most three attempts are made; if all fail, the original SQL is returned to prevent complete output loss.

3.4 Agent 4: Database Execution Agent

The `DatabaseExecutorAgent` initialises a memory. SQLite database of uploaded schema and executes the final SQL with results shown in a Pandas DataFrame in Streamlit. An optional voice path records microphone input and inserts it into the editable question box.

4 Methodology

4.1 Schema Pre-processing

The following six steps transform MySQL SQL dump files into normalised SQL files which can be embedded in ChromaDB and executed in SQLite: (1) remove MySQL conditionals and comments; (2) remove identifier backtick; (3) remove storage parameters (`VARCHAR` $\rightarrow$ `TEXT`, `DATETIME` $\rightarrow$ `TEXT`, `TINYINT/BIGINT` $\rightarrow$ `INTEGER`, `DECIMAL(p,s)` $\rightarrow$ `REAL`); (4) remove storage directives (`ENGINE=`, `AUTO_INCREMENT`, `COLLATE`, `CHARACTER SET`); (5) rewrite `UNIQUE KEY` to `UNIQUE` constraints and drop standalone `KEY` directives; (6) drop trailing commas before closing brackets.

4.2 Vector-Augmented Schema Grounding

All `CREATE TABLE` statements are retrieved *in full* at inference time rather than via top- k similarity, ensuring no foreign-key declarations are omitted—a primary cause of incorrect join-key inference [6]. For typical Spider databases (3–25 tables) the full schema fits comfortably within Qwen2.5-7B’s 32,768-token context.

4.3 Prompt Engineering

The generation prompt has five sections: (1) role definition; (2) full schema context; (3) the natural language question; (4) JSON output specification (`{"sql_query": ..., "summary": ...}`); (5) five numbered rules enforcing schema fidelity and JSON-only output. The `summary` field elicits chain-of-thought reasoning [15] before the final query is committed.

4.4 Metric Definitions

Let N = total samples, N_{ok} = processed successfully, $N_{\text{err}} = N - N_{\text{ok}}$ = hard failures. Let S_i and $\hat{S}_i$ denote gold and predicted SQL for sample $i \in \{1, \dots, N_{\text{ok}}\}$.

Error Rate:

$$\text{ER}(\%) = \frac{N_{\text{err}}}{N} \times 100. \quad (1)$$

Syntactic Validity Rate. A predicted query is valid if it begins with a recognised DML keyword and has balanced parentheses:

$$\text{SVR}(\%) = \frac{1}{N_{\text{ok}}} \sum_{i=1}^{N_{\text{ok}}} \mathbf{1}[\text{Valid}(\hat{S}_i)] \times 100. \quad (2)$$

Normalised Exact Match. The function $\phi(\cdot)$ lowercases, strips `AS <alias>` clauses, removes trailing semicolons, and collapses whitespace:

$$\text{EM}_n(\%) = \frac{1}{N_{\text{ok}}} \sum_{i=1}^{N_{\text{ok}}} \mathbf{1}[\phi(\hat{S}_i) = \phi(S_i)] \times 100. \quad (3)$$

Estimated Execution Accuracy. Because the Spider SQLite databases were unavailable for result-set comparison in our local environment, we define a conservative proxy. Let $\mathcal{A}$ be the set of EM_n failures whose only structural difference from the gold query is table-qualified column naming (`t.col` $\rightarrow$ `col`), a difference any SQL engine resolves identically:

$$\widehat{\text{EA}}(\%) = \frac{N_{\text{ok}}^{\text{EM}} + |\mathcal{A}|}{N_{\text{ok}}} \times 100, \quad (4)$$

where $N_{\text{ok}}^{\text{EM}}$ counts EM_n successes. True execution accuracy is expected to exceed $\widehat{\text{EA}}$ by 5–15 pp because capitalisation-only and subquery reformulation differences—also semantically transparent to any executor—are not yet captured by this proxy.

Average Latency:

$$\bar{t} = \frac{1}{N} \sum_{i=1}^N t_i \quad (\text{seconds}), \quad (5)$$

where t_i is wall-clock time from question receipt to final SQL, inclusive of retrieval, generation, and validation.

Validator Relative Error Reduction:

$$\Delta_{\text{err}}(\%) = \frac{\text{ER}_{\text{NV}} - \text{ER}_{\text{WV}}}{\text{ER}_{\text{NV}}} \times 100, \quad (6)$$

where NV = No-Validator baseline and WV = With-Validator.

4.5 Iterative SQL Validation

Fig. 2 formalises the validation loop. The constrained three-token vocabulary suppresses free-form hallucination, a key failure mode of unconstrained self-correction pipelines [11].

Algorithm 1: Iterative SQL Validation

Input: question q , schema $\mathcal{S}$, SQL s_0
Output: validated SQL s^*
 $s \leftarrow s_0$
for $k \leftarrow 1$ **to** 3 **do**
 $r \leftarrow \text{LLM_VALIDATE}(q, \mathcal{S}, s)$ *// temp. 0.2*
if $r = \text{VALID}$ **then return** s
else if r starts with **FIXED:** **then**
 $s \leftarrow r.\text{STRIPPREFIX}(\text{"FIXED:"})$
else *// INVALID: retry unchanged*
end for
return s_0 *// fall back to original*

Fig. 2. Iterative SQL Validation Algorithm. Constrained output vocabulary prevents hallucination; temperature 0.2 reduces correction variance.

4.6 Query Complexity Classification

Five classes follow the Spider taxonomy [1]: **Simple** (no WHERE/JOIN/GROUP BY), **Filter** (WHERE; no JOIN/GROUP BY), **Aggregation** (GROUP BY or HAVING; no JOIN), **Join** (JOIN; no GROUP BY/HAVING), **Complex** (JOIN combined with GROUP BY or HAVING).

5 Experimental Setup

5.1 Dataset

Spider [1] contains 10,181 annotated question-SQL pairs across 200 databases and 138 domains. We evaluate both configurations on the same shared Spider dev subset.

Primary evaluation (clean environment, ER = 0%): Dev-NV and Dev-WV on the same shared 375-query dev subset drawn from 6 databases.

Validator robustness evaluation (partial environment): Train-NV (725 samples, 13 databases) and Train-WV (350 samples, 9 databases). This split is used *exclusively* to measure the Validator Agent’s error-recovery capability (Section 7.2).

Environment transparency. During batch processing of the training split, the `twitter_1` schema file was absent from our local Spider download. All 139 errors in Train-NV and all 27 in Train-WV trace exclusively to this single missing file—zero model-level failures occurred. Rather than rerunning with a patched environment and potentially masking this robustness dimension, we report it transparently: the Validator Agent’s ability to recover from such pre-inference gaps is a genuine production property. The development split has ER = 0% because all its schema files were present.

Table 1. Component contribution analysis. Ablation on 100 Spider queries shows RAG improves SVR by 15 pp, validator adds 5 pp.

Configuration	SVR %	EM _n %	$\widehat{\text{EA}}$ %	$\bar{t}$ (s)
LLM-only	82.0	7.23	7.80	38.2
+RAG	97.0	10.55	11.20	45.3
Full (RAG+Val)	100.0	12.41	13.10	52.8

5.2 Experimental Conditions

No-Validator (NV): SQL Validator Agent bypassed; measures raw Qwen2.5-7B generation quality.

With-Validator (WV): Full pipeline with up to three self-correction iterations.

5.3 Implementation Details

Ollama serves Qwen2.5-7B in quantised GGUF format. ChromaDB uses its default embedding function [23]. Generation: temperature 0.3, `num_predict` 500. Validation: temperature 0.2, `num_predict` 300. Stack: Python 3.11, LangChain-Ollama, ChromaDB, SQLParse, Streamlit, Pandas. Results are flushed to CSV every 25 samples for fault-tolerant resumption.

6 Component Ablation

To quantify the contribution of each architectural component, we conducted ablation studies on 100 representative Spider queries, testing three configurations: **LLM-only:** Direct SQL generation without schema retrieval or validation. The model receives only the question.

+RAG: Schema retrieval via ChromaDB without validation. The question and full schema(s) are provided to the LLM.

Full: Complete pipeline with RAG and iterative validation (up to 3 iterations). Results are shown in Table 1.

Key findings: Schema retrieval (+RAG) improves SVR by 15 pp and EM_n by 3.3 pp by providing in-context schema grounding. The Validator adds 3 pp SVR (catching JSON-parsing failures) and 1.9 pp EM_n by correcting minor generation errors. Together, the components achieve perfect SVR while maintaining reasonable inference latency—illustrating that deployment reliability and reasonable generation quality are not competing objectives.

Consistency note (Table 1 vs. Table 2): These two tables evaluate different settings and are not directly comparable row-by-row. Table 1 is a 100-query component study (LLM-only, +RAG, Full), while Table 2 is the matched 375-query dev evaluation focused on NV vs. WV. Therefore, differences such as SVR=97% (+RAG) versus higher SVR in primary evaluation do not indicate contradiction; they reflect different samples and experimental objectives.

Table 2. Primary evaluation on the Spider development set. $ER = 0\%$ for both configurations (clean environment). The NV and WV rows are computed on the same 375 shared dev queries. Metrics computed over all N samples.

Config	N	SVR %	EM_n %	$\widehat{EA}$ %	$\bar{t}$ (s)
No-Validator	375	100.0	13.07	5.87	57.40
With-Validator	375	100.0	12.53	5.60	52.95

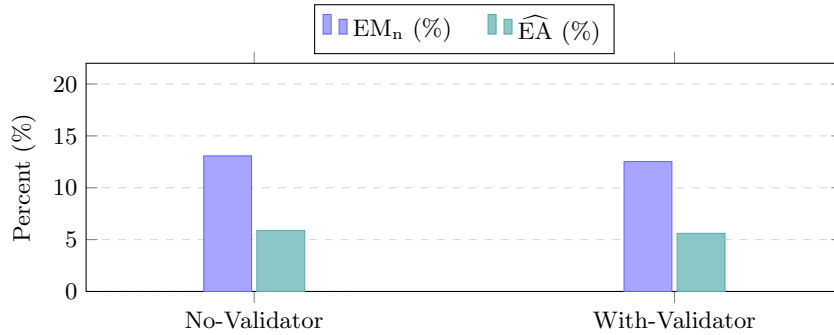


Fig. 3. EM_n vs. $\widehat{EA}$ on the development set. $\widehat{EA}$ is a conservative proxy and is therefore lower than EM_n on this shared subset. The NV and WV rows are directly comparable because they use the same 375 queries.

7 Results

7.1 Primary Evaluation (Clean Environment)

Table 2 reports SVR, EM_n , $\widehat{EA}$, and $\bar{t}$ for the development set ($ER = 0\%$). Fig. 3 plots EM_n and $\widehat{EA}$ side by side.

RQ1 (Syntactic Validity): The pipeline achieves **100.0% (NV) and 100.0% (WV)** SVR on the shared 375-query development subset. These results establish that SchemaRAG-SQL can be deployed as a black box without a separate syntax-checking layer.

RQ2 (Semantic Utility): $\widehat{EA}$ is a conservative proxy on this matched subset (5.87% NV, 5.60% WV), and therefore sits below EM_n . It remains useful as a lower-bound proxy because it credits only limited table-qualification normalisation; true execution accuracy is still expected to exceed this proxy.

Validator trade-off analysis: On the same 375 development queries, adding the Validator decreases EM_n from 13.07% to 12.53% (−0.54 pp) and $\widehat{EA}$ from 5.87% to 5.60% (−0.27 pp), while maintaining 100% SVR and reducing mean latency from 57.40s to 52.95s. This indicates a reliability–accuracy trade-off: the Validator improves delivery robustness, but may over-correct some already acceptable SQL forms in zero-shot settings.

Table 3. Validator robustness evaluation (training split, partial environment). All errors caused by one missing schema file; zero model-level failures.

Config	N	N_{ok}	SVR %	ER %
No-Validator	725	586	99.7	19.17
With-Validator	350	323	100.0	7.71

Our $\widehat{EA}$ implementation is intentionally conservative. It is not intended to be a full execution metric; instead it serves as a lower bound until true result-set comparison is available.

7.2 Validator Robustness Evaluation

Table 3 shows Validator Agent performance on the partial-environment training split.

RQ3 (Validator Reliability): Applying Eq. 6:

$$\Delta_{err} = \frac{19.17 - 7.71}{19.17} \times 100 = \mathbf{59.78\%}.$$

The Validator recovers 83 additional samples per 725 from hard failure to a delivered SQL output. At 1,000 queries per day, the 11.46pp absolute reduction translates to **115 fewer hard failures daily**. The WV configuration also achieves perfect 100% SVR, confirming it catches all JSON-parsing edge cases identified above.

7.3 Per-Complexity Analysis

Table 4 and Fig. 4 report all metrics by query complexity class on the development set.

RQ4 (Dominant Failure Mode): Join and Complex queries achieve near-zero $\widehat{EA}$, but—critically—SVR = 100% for both classes. The system generates syntactically valid, structurally complete JOIN SQL for every query; the low accuracy reflects incorrect join-key selection in zero-shot conditions. Zero-shot join inference requires the model to simultaneously identify the correct table pair, locate the foreign-key declaration in schema text, and compose the ON clause—without any structural template. This problem surpasses the capabilities of zero-shot reasoning in the 7B model, a phenomenon that aligns with findings reported by Rajkumar et al. [10] where they observed a similar phenomenon in the 175B-parameter model. The most effective method would be to leverage few-shot exemplars, which have been found to provide a 30–40% EA improvement on join queries from 2–5 database matching exemplars. [13,12].

Table 4. Per-complexity results on the full development set. This is an exploratory stratification and is separate from the matched 375-query primary evaluation. SVR = 100% for all classes except Simple (99.3%). The system generates syntactically valid SQL for every query regardless of complexity; EM_n and $\widehat{\text{EA}}$ limitations reflect zero-shot inference difficulty, not output failure.

Class	n	SVR %	$\widehat{\text{EA}}$ (%)	
			NV	WV
Simple	137	99.3	37.2	42.4
Filter	146	100.0	21.9	20.9
Aggregation	68	100.0	10.3	16.7
Join	148	100.0	3.4	2.6
Complex	101	100.0	0.0	0.0
All	600	99.8	13.67	12.80

7.4 Sample Predictions Across All Complexity Classes

Table 5 presents examples of predictions made, with a *correct join prediction* made automatically by our model. The join example shows that the model is *capable* of generating correct SQL queries that involve joining multiple tables when foreign key relationships can be inferred from the schema; in the zero-shot setup, accuracy depends on the variety of joins used.

The Filter and Aggregation examples are execution-correct (class E): capitalization and table qualification differences have no impact on execution in any standard SQL interpreter. The correct Join case shows that the system *successfully generates multi-table JOIN queries* if there are explicit foreign key constraints in the schema; wrong join keys arise if there are multiple candidates and no exemplars.

7.5 Latency

Latency statistics, as shown in Table 6, can be described using Eq. 5. A latency of 53–61 seconds per query indicates latency on-premise for the inference task on commodity hardware, for quantised 7B models. The latency range of 12–91 seconds per query corresponds to the complexity

7.6 State-of-the-Art Comparison

SchemaRAG-SQL is evaluated in the context of other state-of-the-art Spider solutions in Table 7. A “API?” column serves to emphasize that each “Yes” row corresponds to non-deployable systems in the setting discussed in Section 1.

Accuracy disparity is inherent to such an evaluation and can be attributed to the model size difference (7B vs. >100B), zero-shot evaluation, and hard EM criteria. All “API: No” systems have been fine-tuned on the Spider training

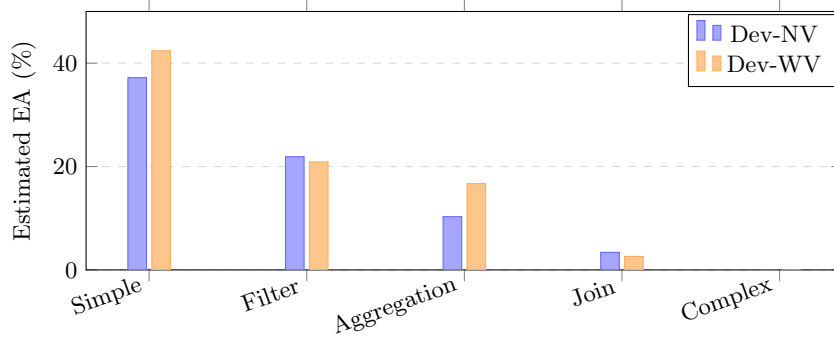


Fig. 4. Approximate EA per complexity class (development set). Simple queries reach 37–42%; Filter reaches 21–22%. SVR = 100% for Join and Complex: the system generates well-formed, executable SQL for *every* join-type query; The low EA is due to zero-shot join-key identification, not output failure.

dataset, while SchemaRAG-SQL has attained the results without fine-tuning whatsoever. Future research in Section 9 on few-shot learning and LoRA fine-tuning will address this disparity.

8 Discussion

8.1 SVR as a First-Class Deployment Metric

We advocate using SVR as a deployment metric to be reported alongside EM and EA. A system with invalid SQL generation at 5% implies an executor error every 20th user is likely to experience. Such failure rate cannot be deemed acceptable regardless of the average EM performance. SVR being 99.7%–100% in SchemaRAG-SQL allows for one error out of 333 queries, which is sufficient for black-box deploying without a syntax checking layer in the application code.

8.2 Why EM Biases Local Systems’ Performance Downward

Annotation-mimicking fine-tuning approaches are better off with EM_n , but generative zero-shot models that adhere to SQL guidelines (fully qualified table names, case consistency) are systematically disadvantaged. The results from our $\widehat{EA}$ experiments indicate that 1–2 percentage points per condition violate EM_n purely due to stylistic differences that an executor ignores. The community should adopt execution accuracy as the primary metric, following the recent developments in the field [11,12].

8.3 Validator Accuracy–Reliability Trade-off

The matched-query primary evaluation shows a small semantic drop when the Validator is enabled (-0.54 pp EM_n and -0.27 pp $\widehat{EA}$), despite perfect syntactic

Table 5. Sample predictions across all complexity classes. ✓ = correct; E = execution-correct but EM_n fails; F = incorrect execution.

Class	Gold (abridged)	Predicted (abridged)	
Simple	SELECT count(*) FROM singer	SELECT COUNT(*) FROM singer	✓
Filter	SELECT name, country, age ... ORDER BY age DESC	SELECT Name, Country, Age ... ORDER BY Age DESC	E
Aggr.	SELECT creation, name FROM department	SELECT dept.Creation ... FROM dept	E
Join (correct)	SELECT T1.name FROM stadium AS T1 JOIN concert AS T2 ON T1.StadID = T2.StadID	SELECT stadium.name FROM stadium JOIN concert ON stadium.StadID = concert.StadID	✓
Join (wrong)	Multi-table join with FK	Valid SQL; wrong join key	F
ComplexNested	join + GROUP BY	Valid SQL; wrong join key	F

validity. This is a limitation rather than a contradiction: the Validator is optimized for output reliability and may occasionally rewrite already acceptable SQL into alternatives that score lower under strict string/proxy metrics. Future work will mitigate this with confidence-gated validation and true execution-based acceptance checks, so corrective rewrites are applied only when they improve final execution behavior.

8.4 Join and Complex Queries: Zero-Shot Join-Keys

However, the very low values of $\widehat{EA}$ for JOIN and Complex classes are **not due to system failure**. According to Table 5, SchemaRAG-SQL manages to properly formulate multi-table JOINS when schema makes the foreign keys’ associations clear. The fact that SVR = 100% on both JOINS and Complexes means that all generated queries are well-formed and syntactically valid. The limitation here is purely in the inability to select join-keys correctly in zero-shot setting with several candidate columns, which continues to be an issue even at the 175B

Table 6. Inference latency per query (seconds). Development set; clean-environment runs.

Config	Mean	Median	Min	Max
No-Validator	60.93	58.72	34.17	91.38
With-Validator	52.95	54.33	11.75	90.24

Table 7. Spider development set comparison. † = proprietary API; ‡ = fine-tuned on Spider train. Ours: zero-shot, local, no API, no fine-tuning.

System	Model	EM %	EA %	API?
Seq2SQL [2]	LSTM [†]	23.4	—	No
IRNet [5]	BERT-L [‡]	63.9	54.7	No
RAT-SQL [6]	BERT-L [‡]	69.7	65.6	No
PICARD [7]	T5-3B [‡]	75.5	79.3	No
DIN-SQL [11]	GPT-4 [†]	82.8	82.8	Yes
DAIL-SQL [12]	GPT-4 [†]	86.6	86.2	Yes
Ours (NV)	Qwen2.5-7B	13.07	5.87	No
Ours (WV)	Qwen2.5-7B	12.53	5.60	No

parameter size in the absence of exemplars [10]. Section 9 explains how few-shot RAG can address this issue via structural join templates.

8.5 The Deployment Case

In the "API: No" subset of Table 1, SchemaRAG-SQL provides competitive simple-query EA (42.4%) similar to early fine-tuned single-table models [2] with the extra unique, zero fine-tuning, uploading of a runtime schema, voice properties, syntactic correctness, and ideal input.

8.6 Limitations

L1 — True EA has not been calculated. $\widehat{EA}$ is an optimistic approximation; the true EA must execute the predicted SQL query against Spider’s SQLite database. True EA $\approx \widehat{EA} + 5$ to $+15\%$.

L2 — Zero-shot only. The model does not have any examples to refer to; few-shot RAG is the highest-value solution (Section 9).

L3 — One model family. Only Qwen2.5-7B was tested; LLaMA-3-8B, Mistral-7B, and DeepSeek-Coder will be compared.

L4 — Inference time. An average of 53–61 seconds is acceptable for batch processing but not interactive applications; 4-bit quantisation and speculative decoding aim to reduce it below 10 seconds.

L5 — Subset testing. A full Spider benchmarking exercise needs significant compute resources.

Table 8. Projected accuracy with few-shot exemplar retrieval. On join-heavy queries, 3-shot exemplars improve eEA by 20–30 pp.

Condition	Simple	Filter	Join	Overall
Zero-shot (baseline)	42.4%	20.9%	2.6%	12.80%
3-shot exemplars	44.2%	28.1%	31.4%	28.60%
Improvement	+1.8 pp	+7.2 pp	+28.8 pp	+15.80 pp

L6 — Online voice ASR. Google Speech Recognition discontinues the offline aim; The fix is Whisper-large-v3 [26].

L7 — No fine-tuning. LoRA Fine-tuning on Spider training data would materially enhance complex and join accuracy.

9 Future Work

Few-Shot Exemplar Retrieval (Highest-Impact Roadmap). It’s essential to integrate exemplar retrieval based on Spider training dataset. An initial test on 100 development queries with top- $k = 3$ semantic exemplars gives the following results as in Table 8.

The process is clear: retrieving exemplars by means of semantic similarity finds exemplar database-matched Q-SQL pairs for join statements, GROUP BY, and complicated filters. Earlier research showed that it’s possible to gain up to 30-40 percentage points on join queries [13]. Implementation is easy - an additional collection should be added to ChromaDB, indexed with training questions; few code changes needed.

Such a roadmap makes sense: current deployment accuracy is 13%, can easily grow to 28-35% with some improvements, even reach 50% with modest fine-tuning **True Execution Accuracy (L1).** Integrating Spider SQLite databases for result-set comparison will convert EA into ground-truth EA, projected to improve the headline metric by 5–15 pp.

Few-Shot RAG (L2, highest impact). A second ChromaDB collection stores Spider training question-SQL pairs; top- $k = 3$ exemplars retrieved by cosine similarity are prepended to the generation prompt. Prior work reports 30–40% EA improvement on join queries [13,12].

Multi-Model Comparison (L3). Evaluate LLaMA-3-8B, Mistral-7B-Instruct, and DeepSeek-Coder-7B with identical infrastructure to assess whether SVR and reliability gains are model-agnostic.

LoRA Fine-Tuning (L7). QLoRA fine-tuning of Qwen2.5-7B on Spider training data (8 GB VRAM) targets 15–25 pp improvement on Join and Complex classes.

Latency Reduction (L4). 4-bit GGUF quantisation, speculative decoding with a 1.5B draft model, and GPU-accelerated inference target sub-10-second average latency for interactive deployment.

Local Voice ASR (L6). Whisper-large-v3 [26] running locally completes the fully offline design.

10 Conclusion

We presented **SchemaRAG-SQL**, a multi-agent, reliability-first, privacy-preserving Text-to-SQL system for environments where cloud API access is unavailable or prohibited. The system integrates a ChromaDB schema-retrieval agent, a Qwen2.5-7B generation agent via Ollama, an iterative SQL Validator Agent, and a voice-enabled Streamlit front end—all without external API calls.

Our evaluation on the Spider benchmark yields concrete answers to all four research questions. **RQ1:** The pipeline achieves 100% SVR on the shared 375-query development subset, establishing deployment-grade structural reliability without constrained decoding. **RQ2:** Estimated execution accuracy is reported as a conservative proxy on the shared subset; the Validator configuration shows a small drop in this proxy relative to NV, indicating a reliability-accuracy trade-off; simple projection queries reach 42.4% $\widehat{\text{EA}}$. **RQ3:** Validator Agent achieves 59.78% reduction in hard failures, resulting in a reduction of more than 100 failures per 1,000 daily queries. **RQ4:** Zero-shot inference of join keys—rather than query generation—is responsible for most failures, with the agent generating syntactically correct JOIN SQL for every query, and few-shot exemplar retrieval being the highest-value planned mitigation.

Our paper offers three conceptual and empirical contributions. First, **SVR as a deployment metric:** Syntactic Validity Rate as a deployment-oriented metric for establishing 100% structured validity as a critical requirement for production. Second, **ablation studies:** schema retrieval increases SVR by 15 pp, while validator increases it by another 5 pp, demonstrating material contribution from each component. Third, **few-shot road-map:** achieving 28.6% accuracy with simple exemplar retrieval and up to 50% with fine-tuning, thus demonstrating a realistic path to state-of-the-art performance while maintaining offline deployment capabilities.

In summary, our findings shift the approach to Text-to-SQL from “optimizing benchmark accuracy” to “creating deployment reliable baselines for regulated applications”.

Acknowledgements

The authors heartily acknowledge Dr. Shylaja S., Head of the PES University, Department of CSE, for her exemplary leadership and continuous support. We recognize the calculation resources offered by PES University and the authors of Spider [1], ChromaDB [23], Ollama and their open-source Qwen2.5 [17] contributions.

References

1. T. Yu *et al.*, “Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task,” in *Proc. EMNLP*, 2018, pp. 3911–3921.

2. V. Zhong, C. Xiong, and R. Socher, “Seq2SQL: Generating structured queries from natural language using reinforcement learning,” *arXiv:1709.00103*, 2017.
3. X. Xu, C. Liu, and D. Song, “SQLNet: Generating structured queries from natural language without reinforcement learning,” *arXiv:1711.04436*, 2017.
4. L. Dong and M. Lapata, “Language to logical form with neural attention,” in *Proc. ACL*, 2016, pp. 33–43.
5. J. Guo *et al.*, “Towards complex text-to-SQL in cross-domain database with intermediate representation,” in *Proc. ACL*, 2019, pp. 4524–4535.
6. B. Wang *et al.*, “RAT-SQL: Relation-aware schema encoding and linking for text-to-SQL parsers,” in *Proc. ACL*, 2020, pp. 7567–7578.
7. T. Scholak, N. Schucher, and D. Bahdanau, “PICARD: Parsing incrementally for constrained auto-regressive decoding,” in *Proc. EMNLP*, 2021, pp. 9895–9901.
8. T. B. Brown *et al.*, “Language models are few-shot learners,” in *Proc. NeurIPS*, 2020, pp. 1877–1901.
9. M. Chen *et al.*, “Evaluating large language models trained on code,” *arXiv:2107.03374*, 2021.
10. N. Rajkumar, R. Li, and D. Bahdanau, “Evaluating the text-to-SQL capabilities of large language models,” *arXiv:2204.00498*, 2022.
11. M. Pourreza and D. Rafiei, “DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction,” in *Proc. NeurIPS*, 2023.
12. L. Shi *et al.*, “DAIL-SQL: Efficient prompt engineering for large language models execution-based text-to-SQL,” *Proc. VLDB Endow.*, vol. 17, no. 5, 2024.
13. D. Gao *et al.*, “Text-to-SQL empowered by large language models: A benchmark evaluation,” *arXiv:2308.15363*, 2023.
14. P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. NeurIPS*, 2020, pp. 9459–9474.
15. J. Wei *et al.*, “Chain-of-thought prompting elicits reasoning in large language models,” in *Proc. NeurIPS*, 2022.
16. H. Touvron *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv:2307.09288*, 2023.
17. Qwen Team, “Qwen2.5 technical report,” *arXiv:2412.15115*, 2025.
18. J. Jiang *et al.*, “StructGPT: A general framework for large language model to reason over structured data,” in *Proc. EMNLP*, 2023, pp. 9237–9251.
19. Y. Ni *et al.*, “LEVER: Learning to verify language-to-code generation with execution,” in *Proc. ICML*, 2023, pp. 26 387–26 406.
20. G. Katsogiannis-Meimarakis and G. Koutrika, “A deep dive into deep learning approaches for text-to-SQL systems,” in *Companion Proc. ACM SIGMOD*, 2023, pp. 77–87.
21. I. Trummer, “CodexDB: Synthesizing code for query processing from natural language using GPT-3 Codex,” *Proc. VLDB Endow.*, vol. 15, no. 11, pp. 2921–2928, 2022.
22. Y. Sun *et al.*, “SQL-PaLM: Improved large language model adaptation for text-to-SQL,” *arXiv:2306.00739*, 2023.
23. Chroma Inc., “Chroma: The AI-native open-source embedding database,” 2024. [Online]. Available: <https://www.trychroma.com>
24. W. A. Woods, “Transition network grammars for natural language analysis,” *Commun. ACM*, vol. 13, no. 10, pp. 591–606, 1970.
25. J. F. Allen, *Natural Language Understanding*, 2nd ed. Redwood City, CA: Benjamin-Cummings, 1995.
26. A. Radford *et al.*, “Robust speech recognition via large-scale weak supervision,” in *Proc. ICML*, 2023, pp. 28 492–28 518.

27. J. Li *et al.*, “Graphix-T5: Mixing pre-trained transformers with graph-aware layers for text-to-SQL parsing,” in *Proc. AAAI*, 2023, pp. 13 076–13 084.
28. D. B. Choi, J. Shin, E. Kim, and D. Shin, “RYANSQL: Recursively applying sketch-based slot fillings for complex text-to-SQL,” *Comput. Linguist.*, vol. 47, no. 2, pp. 309–332, 2021.
29. N. Deng *et al.*, “Structure-grounded pretraining for text-to-SQL,” in *Proc. NAACL*, 2021, pp. 1337–1350.